

MÓDULO PROYECTO

CURSO 2025-2026

TÍTULO:

**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN,
AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

ALUMNO: Diego Rodríguez López

CICLO FORMATIVO: Desarrollo de Aplicaciones Web

TUTOR: Sonia Sagrario Mastache Maestre

CENTRO: [PENDIENTE: nombre del centro educativo]

[PENDIENTE: fecha de entrega definitiva]

RESUMEN

El presente proyecto documenta el desarrollo de LAVANDERÍA KWL, un sistema integral para la gestión, automatización y control de una lavandería autoservicio. El proyecto surge de una necesidad real: sustituir progresivamente un sistema propietario cerrado por una solución propia, mantenible y libre de dependencia externa, capaz de controlar máquinas, registrar ciclos, gestionar créditos, consultar caja, visualizar cámaras y operar elementos de domótica como puerta, luces y ventilación. La solución se ha diseñado mediante una arquitectura distribuida compuesta por frontend web, backend Node.js con TypeScript, base de datos MariaDB, caché Redis, broker MQTT Mosquitto, simulador de dispositivos y despliegue mediante Docker Compose. El sistema permite operar lavadoras y secadoras reales o simuladas, registrar eventos técnicos, auditar acciones administrativas y mantener trazabilidad económica de ciclos, ampliaciones y bonificaciones. El backend centraliza la lógica de negocio y actúa como punto de decisión; los dispositivos, físicos o simulados, solo ejecutan órdenes y publican estado mediante MQTT. El panel web ofrece vistas diferenciadas para máquinas, caja, informes, usuarios, cámara, ajustes, inicio, domótica y logs. La memoria incluye análisis de requisitos, estimación de costes, tecnologías utilizadas, diseño de interfaz, modelo de base de datos, arquitectura, comunicación IoT, despliegue, pruebas, dificultades y posibles ampliaciones. El resultado es un MVP funcional y ampliable que valida la base técnica para una futura implantación física completa con ESP32, pagos online, app de cliente, notificaciones y mantenimiento predictivo.

ÍNDICE

[PENDIENTE: actualizar el índice automático en Word antes de exportar a PDF. Instrucción: seleccionar este bloque, ir a Referencias > Tabla de contenido, o usar clic derecho > Actualizar campo si se inserta una tabla automática.]

RESUMEN [PENDIENTE: página]

ÍNDICE DE FIGURAS [PENDIENTE: página]

Proyecto: LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN,
AUTOMATIZACIÓN Y CONTROL DE UNA

LAVANDERÍA [PENDIENTE: página]

1. Introducción [PENDIENTE: página]

1.1. Descripción del objeto de estudio; a quién va dirigida; qué existe hasta ahora; etc. [PENDIENTE: página]

1.2. Objetivos [PENDIENTE: página]

1.3. Alcance, requisitos y criterios de éxito del MVP
[PENDIENTE: página]

2. Análisis, diseño y desarrollo [PENDIENTE: página]

2.1. Estimación de costes. Tiempo invertido.

Justificación. [PENDIENTE: página]

2.2. Tecnologías y herramientas utilizadas
[PENDIENTE: página]

2.3. Diseño de interfaz [PENDIENTE: página]

2.4. Concepción de la base de datos [PENDIENTE:
página]

2.5. Diseño y arquitectura de la solución [PENDIENTE:
página]

2.6. Principales dificultades encontradas en el desarrollo del proyecto y cómo se han afrontado [PENDIENTE: página]

2.7. Despliegue [PENDIENTE: página]

2.8. Validación y pruebas [PENDIENTE: página]

3. Conclusiones y posibles ampliaciones [PENDIENTE:
página]

4. Bibliografía y webgrafía [PENDIENTE: página]

Anexos [PENDIENTE: página]

ÍNDICE DE FIGURAS

Figura 1. Diagrama de red del sistema KWL	
[PENDIENTE: página]	
Figura 2. Diagrama UML de componentes	[PENDIENTE: página]
Figura 3. Diagrama UML de despliegue	[PENDIENTE: página]
Figura 4. Modelo entidad-relación del sistema	
[PENDIENTE: página]	
Figura 5. Diagrama de estados de máquina	
[PENDIENTE: página]	
Figura 6. Diagrama de secuencia de concesión de crédito	[PENDIENTE: página]
Figura 7. Captura de la página pública de inicio	
[PENDIENTE: página]	
Figura 8. Captura del panel de máquinas	[PENDIENTE: página]
Figura 9. Captura del panel de caja e informes	
[PENDIENTE: página]	
Figura 10. Captura del simulador IoT/MQTT	
[PENDIENTE: página]	
Figura 11. Captura del despliegue Docker Compose	
[PENDIENTE: página]	
Figura 12. Captura de comprobación de health del backend	[PENDIENTE: página]

Proyecto: LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA

1. Introducción

1.1. Descripción del objeto de estudio; a quién va dirigida; qué existe hasta ahora; etc.

El objeto de estudio de este proyecto es una plataforma de gestión, automatización y control para una lavandería autoservicio real. La solución se denomina LAVANDERÍA KWL y se concibe como un sistema completo compuesto por aplicación web, backend, base de datos, mensajería MQTT, infraestructura local y capa IoT. Su finalidad es permitir que el propietario del negocio controle la operación diaria de la lavandería sin depender de un proveedor externo para tareas críticas de mantenimiento, configuración y explotación de datos.

El proyecto no se limita a una página web administrativa. El sistema tiene que interactuar con elementos físicos de la lavandería: lavadoras, secadoras, monederos, relés, puerta motorizada, iluminación, ventilación y cámaras. Para poder probar el comportamiento sin depender siempre de hardware físico, también se incluye un simulador capaz de publicar estados y eventos MQTT de máquinas y dispositivos. De esta forma, la arquitectura puede validarse en un entorno de evaluación antes de pasar a una integración física definitiva.

La solución está dirigida a varios perfiles. El primero es el administrador o propietario, que necesita consultar el estado de las máquinas, aplicar créditos, reiniciar o detener máquinas, consultar caja, generar informes, administrar usuarios y modificar parámetros operativos. El segundo perfil es el operador o empleado, que puede necesitar funciones limitadas para supervisar la tienda o resolver incidencias. El tercer perfil es el técnico mantenedor, que debe contar con información de logs, auditoría y estado de comunicaciones. Como ampliación futura, el sistema podría incorporar acceso para clientes finales, de modo que puedan consultar el estado de sus lavados, recibir avisos o realizar pagos desde una aplicación.

La situación inicial parte de un sistema propietario ya instalado en la lavandería. Según la entrevista y la consultoría técnica realizadas, el sistema existente permite funciones como monitorización básica de máquinas, control de crédito, apagado o reinicio, visión de cámaras, control de puerta y consulta de contabilidad. Sin embargo, el propietario no dispone de acceso total al software, a la base de datos ni al código de los módulos. Además, el estado mostrado por el sistema actual no siempre representa el funcionamiento real de la máquina, ya que en algunos casos se basa en un cronómetro estimado y no en una lectura completa de fallos, pausas o apertura de puerta (Rodríguez López, 2026a, 2026b).

La consultoría del sistema actual identifica un entorno formado por monederos RM5, placas personalizadas, comunicación por RJ45, centralita IoT, pulsos de 24

V y relés para acciones como reinicio, apagado y cierre de puerta. Esta información condiciona el diseño del nuevo sistema, que debe ser realista con las limitaciones del hardware existente y con la falta de documentación abierta del proveedor anterior (Rodríguez López, 2026c).

Por tanto, el proyecto se plantea como una sustitución progresiva. En primer lugar se desarrolla un MVP funcional con backend, frontend, base de datos, MQTT, simulación, informes y despliegue. En segundo lugar, se deja preparada la integración con módulos físicos basados en ESP32 u otros microcontroladores compatibles. Esta decisión evita bloquear el desarrollo por la falta de acceso al sistema propietario y permite demostrar el flujo completo de negocio desde la web hasta el dispositivo.

1.2. Objetivos.

El objetivo general es diseñar y desarrollar una solución propia para gestionar, automatizar y controlar una lavandería autoservicio mediante una arquitectura web e IoT, reduciendo la dependencia de proveedores externos y permitiendo mayor control sobre la operación, el mantenimiento y los datos del negocio.

Los objetivos específicos del proyecto son los siguientes:

- Desarrollar un backend central que concentre la lógica de negocio, la autenticación, la gestión de usuarios, la gestión de máquinas, la auditoría, la caja y los informes.
- Crear una interfaz web responsive que permita operar el sistema desde ordenador, tablet o móvil.
- Diseñar una base de datos relacional en MariaDB para almacenar lavanderías, usuarios, máquinas, tarifas, ciclos, movimientos, logs, auditoría y configuración.
- Integrar comunicación MQTT para desacoplar el backend de los dispositivos físicos o simulados.
- Registrar eventos técnicos y acciones administrativas para mejorar la trazabilidad del sistema.
- Implementar un simulador de lavadoras, secadoras y elementos IoT para validar el comportamiento sin depender de hardware real.
- Definir una infraestructura desplegable mediante Docker Compose y adaptable a un entorno real con Proxmox, máquinas virtuales y contenedores LXC.
- Diseñar el sistema con criterios de seguridad, roles, acceso por VPN y separación de servicios.
- Preparar el proyecto para futuras ampliaciones como pagos online, app de clientes, notificaciones, inteligencia artificial, sensórica y explotación avanzada de datos.

El proyecto también tiene objetivos formativos. Permite aplicar conocimientos de desarrollo web, programación backend, base de datos, seguridad, infraestructura, redes, diseño de interfaces, análisis de requisitos, documentación técnica y despliegue. Esta combinación lo convierte en un proyecto transversal dentro del ciclo formativo de Desarrollo de Aplicaciones Web.

1.3. Alcance, requisitos y criterios de éxito del MVP

El alcance del MVP se ha definido con un criterio práctico: demostrar que la arquitectura completa funciona de extremo a extremo. Por ello, la versión documentada incluye panel web, autenticación, gestión de usuarios, gestión de máquinas, control de créditos, caja, informes, cámara, configuración, domótica, logs, auditoría, simulador IoT, MQTT, Redis, MariaDB y despliegue demo. La integración física definitiva con ESP32 queda preparada conceptualmente, pero debe validarse sobre el hardware real de la lavandería antes de considerarse producción.

El criterio de éxito del MVP es que un usuario autorizado pueda operar desde la web, que el backend valide la acción, que se publique un comando MQTT, que un dispositivo o simulador responda con estado o evento, que el backend registre la información y que el panel refleje el cambio. Este flujo confirma la separación entre decisión, comunicación y ejecución.

Tabla 1. *Requisitos funcionales principales del sistema*

Área	Requisito	Implementación prevista o realizada
Control de máquinas	Visualizar lavadoras y secadoras con estado, tipo, código visible y tiempo restante.	Panel de máquinas conectado a API y base de datos.
Control de máquinas	Aplicar crédito, iniciar, pausar, detener o ampliar tiempo según reglas de negocio.	Endpoints protegidos y comandos MQTT hacia máquina o simulador.
Comunicación IoT	Comunicar backend y dispositivos mediante mensajería ligera desacoplada.	Broker MQTT Mosquitto y topics kwl/maquinas y kwl/iot.
Domótica	Controlar puerta, luces, ventilación y horarios de apertura/cierre.	Rutas IoT, scheduler y configuración por lavandería.
Cámara	Integrar visualización o proxy de cámaras del local.	Rutas de cámara con control de acceso administrativo.
Caja e informes	Consultar caja diaria, semanal, mensual, anual y evolución de ciclos.	Módulos de caja e informes sobre movimientos y ciclos.
Usuarios y roles	Administrar usuarios con roles y acceso a lavanderías.	Tablas usuario/usuario_lavanderia y middleware requireAuth/requireRole.
Auditoría y logs	Registrar acciones administrativas y eventos técnicos.	Tablas auditoria y log_maquina.
Simulación	Probar máquinas y dispositivos sin hardware real.	Servicio de simulación Node.js con MQTT y GUI propia.

Fuente: Elaboración propia.

Tabla 2. *Requisitos no funcionales del sistema*

Requisito	Aplicación en el proyecto
Seguridad	Autenticación por token, roles, CORS controlado, Helmet, hash de contraseña y no exposición de servicios internos fuera de la red/VPN.
Mantenibilidad	Separación entre frontend, backend, base de datos, simulación, despliegue y contexto documental.
Escalabilidad	Modelo preparado para múltiples lavanderías y máquinas asociadas a cada local.

Requisito	Aplicación en el proyecto
Trazabilidad	Registro de auditoría para acciones administrativas y log técnico para eventos de máquinas.
Bajo coste	Uso de software libre y componentes accesibles: Node.js, MariaDB, Redis, Mosquitto, Docker, Proxmox y ESP32.
Tolerancia a fallos	Persistencia histórica en MariaDB y uso de Redis solo como caché operativa, evitando que Redis sustituya a la base de datos.
Usabilidad	Interfaz web responsive con vistas separadas por áreas: máquinas, caja, informes, cámara, usuarios, ajustes e IoT.
Desacoplamiento	El frontend nunca controla hardware directamente; siempre pasa por API y MQTT.

Fuente: Elaboración propia.

Quedan fuera del alcance de la entrega principal los pagos online reales, la app de cliente final, la IA productiva, la explotación multi-local completa, la alta disponibilidad compleja y la integración física definitiva con todos los sensores internos de lavadoras y secadoras. Estos elementos se recogen como ampliaciones futuras para no confundir el MVP con una puesta en producción industrial completa.

2. Análisis, diseño y desarrollo

2.1. Estimación de costes. Tiempo invertido. Justificación.

La estimación de costes se ha planteado distinguiendo entre tiempo de desarrollo, costes técnicos y costes materiales aproximados. Al tratarse de un proyecto académico, la valoración económica es orientativa. Se ha utilizado una tarifa ficticia de 18 €/h para desarrollo general y 22 €/h para tareas de infraestructura o integración IoT, con el fin de reflejar la diferencia de complejidad entre programación web y despliegue de servicios.

Tabla 3. Estimación de tiempo y coste del desarrollo

Actividad	Tiempo estimado	Coste estimado	Justificación
Análisis del sistema actual y entrevista	14 h	252 €	Estudio de necesidades reales, sistema propietario, hardware existente y restricciones de acceso.
Definición de requisitos y alcance del MVP	10 h	180 €	Separación entre funciones actuales, objetivos del cliente y alcance viable para la entrega.
Diseño de arquitectura general	16 h	352 €	Diseño por capas: frontend, backend, MariaDB, Redis, MQTT, simulador e infraestructura.
Diseño de base de datos	14 h	252 €	Modelo conceptual, lógico y físico con tablas para lavanderías, usuarios, máquinas, ciclos y auditoría.
Backend Node.js/TypeScript	55 h	990 €	API, autenticación, rutas, validaciones, cámara, caja, informes, usuarios,

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Actividad	Tiempo estimado	Coste estimado	Justificación
			máquinas y configuración.
Comunicación MQTT e IoT	34 h	748 €	Bridge MQTT, topics, procesado de eventos, comandos, estado runtime y sincronización de tarifas.
Frontend público y administrativo	48 h	864 €	Pantallas públicas, panel admin, CSS responsive, JavaScript de interacción y vistas por módulo.
Simulador de dispositivos	30 h	660 €	Simulación de lavadoras/secadoras, puerta, luces, ventilación, GUI y comunicación MQTT.
Despliegue demo Docker Compose	18 h	396 €	Contenedores MariaDB, Redis, MQTT, backend, frontend, simulador, Adminer y Nginx.
Diseño de infraestructura real	18 h	396 €	Separación VM_CORE, VM_DATA y LXC_MQTT en Proxmox con acceso remoto seguro.
Pruebas, depuración y validación	22 h	396 €	Comprobaciones de API, MQTT, caja, simulador, rutas, logs y despliegue.
Documentación, diagramas y memoria	38 h	684 €	Redacción de memoria, diagramas draw.io, bibliografía APA7 y preparación de entrega.

Fuente: Elaboración propia.

El coste total estimado del trabajo de desarrollo es de 6.170 €. Este importe no representa una factura real, sino una aproximación del valor del tiempo invertido si el proyecto se valorase como encargo profesional. La estimación permite justificar la complejidad del proyecto frente a una aplicación web simple, ya que combina software, comunicaciones, infraestructura, simulación y análisis de hardware.

Tabla 4. Estimación de costes materiales para una implantación real inicial

Elemento	Cantidad orientativa	Coste unitario estimado	Coste estimado	Observaciones
Servidor o miniPC para Proxmox	1	250-450 €	250-450 €	Puede reutilizarse hardware existente si cumple requisitos.
ESP32 o módulo equivalente	1 por máquina/dispositivo	6-15 €	[PENDIENTE]	Necesario para sustitución física progresiva.
Módulo de relés/optoacopladores	1 por máquina	5-20 €	[PENDIENTE]	Debe validarse eléctricamente por las señales de 24 V.
Fuente o convertor 24 V a 5 V/3,3 V	1 por módulo	5-15 €	[PENDIENTE]	Necesario si se alimenta desde la máquina.
Cableado, conectores y caja	Según instalación	Variable	[PENDIENTE]	Depende del local y número de máquinas.
Switch/router/VPN	1	0-120 €	[PENDIENTE]	Puede existir ya en el local.

Elemento	Cantidad orientativa	Coste unitario estimado	Coste estimado	Observaciones
Software base	N/A	0 €	0 €	Se prioriza software libre o gratuito.

Fuente: Elaboración propia.

Para cerrar esta sección antes de la entrega es recomendable sustituir los importes pendientes por costes reales o por presupuestos aproximados de componentes concretos. También conviene adjuntar en anexos las capturas del entorno funcionando y, si existen, fotografías del hardware de pruebas.

2.2. Tecnologías y herramientas utilizadas (nombrarlas y pequeña descripción, justificación de selección)

La elección tecnológica se ha realizado buscando bajo coste, control propio, despliegue local, mantenibilidad y compatibilidad con un sistema IoT. En lugar de depender de una solución cerrada, se ha optado por tecnologías ampliamente documentadas y con comunidad activa.

Tabla 5. Tecnologías principales utilizadas

Tecnología	Uso en el proyecto	Justificación
Node.js	Runtime del backend y del simulador.	Permite crear servidores, herramientas y servicios con JavaScript/TypeScript. Es adecuado para APIs y procesos orientados a eventos (Node.js, s. f.).
TypeScript	Lenguaje principal del backend.	Aporta tipado estático sobre JavaScript, mejorando mantenibilidad y detección temprana de errores (Microsoft, s. f.).
Express	Framework HTTP para la API REST.	Proporciona rutas, middlewares y manejo de peticiones de forma ligera y flexible (OpenJS Foundation, s. f.).
MariaDB	Base de datos relacional.	Permite persistencia estructurada, relaciones, claves foráneas, integridad y consultas SQL (MariaDB, s. f.).
Redis	Caché operativa y apoyo a tiempo real.	Se usa para estado rápido, sin sustituir la persistencia histórica de MariaDB (Redis, s. f.).
MQTT	Comunicación backend-dispositivos.	Protocolo publish/subscribe ligero, apropiado para IoT y comunicaciones M2M (OASIS, 2019).
Eclipse Mosquitto	Broker MQTT.	Broker abierto para MQTT, central en la comunicación entre backend y dispositivos (Eclipse Foundation, s. f.).
WebSocket	Canal de actualización live del panel admin.	Permite enviar snapshots de estado al frontend sin recargar toda la página.
Docker Compose	Despliegue demo multi-contenedor.	Permite definir y ejecutar servicios de la aplicación en un único archivo YAML (Docker, s. f.).
Proxmox VE	Diseño de infraestructura real.	Plataforma de virtualización para separar servicios en VM y LXC (Proxmox Server Solutions GmbH, s. f.).

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Tecnología	Uso en el proyecto	Justificación
		f.).
ESP32	Base prevista para módulos físicos.	Microcontrolador con Wi-Fi/Bluetooth orientado a proyectos IoT (Espressif Systems, s. f.).
WireGuard	Acceso remoto seguro previsto.	VPN moderna y ligera para acceder a la red administrativa sin exponer servicios críticos (WireGuard, s. f.).
OWASP ASVS	Referencia de seguridad.	Proporciona criterios para controles técnicos de seguridad en aplicaciones web (OWASP Foundation, s. f.).

Fuente: Elaboración propia.

El backend del proyecto utiliza dependencias concretas declaradas en package.json. Entre ellas destacan express, cors, helmet, morgan, mqtt, mysql2, bcryptjs y ws. Helmet se utiliza para reforzar cabeceras HTTP; bcryptjs para hash de contraseñas; mysql2 para conexión con MariaDB; mqtt para el bridge con el broker; y ws para WebSocket.

Tabla 6. Dependencias principales del backend

Paquete	Versión	Uso
cors	^2.8.5	Control de orígenes permitidos en la API.
express	^4.19.2	Framework HTTP de la API.
helmet	^7.1.0	Endurecimiento básico de cabeceras HTTP.
morgan	^1.10.0	Registro de peticiones en desarrollo/demo.
mqtt	^5.10.4	Cliente MQTT para publicar comandos y recibir eventos.
mysql2	^3.11.0	Conexión a MariaDB con soporte promise.
bcryptjs	^2.4.3	Hash de contraseñas.
ws	^8.18.0	Servidor WebSocket para actualizaciones en vivo.

Fuente: Elaboración propia.

El simulador también se implementa con Node.js y utiliza express, mqtt y ws. Esta decisión evita duplicar lenguajes y permite que tanto el sistema real como el sistema simulado compartan el mismo modelo conceptual de eventos y comandos.

Tabla 7. Estadística aproximada del código fuente por tecnología

Tecnología	Archivos	Líneas aproximadas
Batch	1	23
CSS	18	6710
HTML	19	2672
JavaScript	16	5463
Markdown	1	187
PowerShell	1	714
SQL	3	815

Tecnología	Archivos	Líneas aproximadas
Shell	7	654
TypeScript	27	5707
YAML	3	234

Fuente: Elaboración propia.

Además de las tecnologías de ejecución, se han utilizado herramientas de apoyo: Visual Studio Code para edición, draw.io/diagrams.net para diagramas, Docker Desktop o Docker Engine para entorno demo, Adminer para inspección de base de datos, terminal Linux/PowerShell para despliegue, y Git como herramienta recomendada de control de versiones.

2.3. Diseño de interfaz (diseño general de pantallas, estructura, paleta de colores, facilidades de uso, elementos destacados, etc.).

La interfaz de LAVANDERÍA KWL se divide en dos zonas: una zona pública orientada a presencia web y una zona administrativa para la operación del negocio. La zona pública incluye páginas como inicio, sobre nosotros, contacto, preguntas frecuentes y textos legales. La zona administrativa incluye máquinas, inicio, caja, informes, cámara, usuarios, ajustes, editor web, IoT y logs.

El diseño general busca claridad y rapidez de uso. En una lavandería autoservicio, el administrador no necesita una interfaz decorativa, sino una herramienta que permita identificar rápidamente qué máquina está libre, en marcha, pausada, fuera de servicio o en mantenimiento. Por ello, la interfaz prioriza tarjetas, tablas, estados visuales y botones de acción directa.

Tabla 8. Pantallas principales de la interfaz

Pantalla	Archivo o módulo	Función principal
Inicio público	public/index.html	Presentación de la lavandería KWL Aqua y acceso general.
Sobre nosotros	public/about.html	Información corporativa del negocio.
Contacto	public/contact.html	Datos de contacto y comunicación con el local.
FAQs	public/faqs.html	Preguntas frecuentes para usuarios.
Máquinas	admin/maquinas.html	Monitorización y control de lavadoras y secadoras.
Inicio admin	admin/inicio.html	Panel operativo general y escenas de apertura/cierre.
IoT	admin/iot.html	Control de luz, puerta, ventilación y programación.
Caja	admin/caja.html	Consulta económica diaria, semanal, mensual y anual.
Informes	admin/informes.html	Análisis de ciclos, tramos horarios y evolución.
Cámara	admin/camara.html	Visualización y control de cámara/proxy.

Pantalla	Archivo o módulo	Función principal
Usuarios	admin/usuarios.html	Gestión de usuarios, roles y acceso a lavanderías.
Ajustes	admin/ajustes.html	Configuración general del sistema.
Logs	admin/logs.html	Consulta de eventos técnicos y auditoría.

Fuente: Elaboración propia.

La estructura visual del panel se basa en navegación persistente, bloques funcionales y separación por módulos. El usuario no tiene que recordar rutas técnicas: accede desde el menú a máquinas, caja, informes, cámara, usuarios, ajustes o logs. Esta organización reduce la carga cognitiva y permite operar el sistema en situaciones reales de mantenimiento o atención al cliente.

La paleta de colores debe reflejar estados operativos. Se recomienda mantener un criterio estable: verde para correcto/disponible, azul para información o máquina en marcha, amarillo para advertencia o pausa, rojo para error/fuera de servicio, gris para inactivo/sin conexión. Aunque el código CSS define la presentación concreta, la memoria deja fijado este criterio para que futuras modificaciones no rompan la coherencia visual.

Como elementos destacados de usabilidad se contemplan: botones de acción visibles, confirmación previa de acciones críticas, mensajes de error comprensibles, actualización de estados, tablas de logs filtrables, estadísticas de caja y panel responsive. Las acciones que pueden afectar al hardware o a la operación del negocio deben permanecer reservadas a usuarios con rol suficiente.

Figura 7. *Captura de la página pública de inicio*

[PENDIENTE: insertar captura real de public/index.html con cabecera, bloque principal y pie de página.]

Fuente: Elaboración propia.

Figura 8. *Captura del panel de máquinas*

[PENDIENTE: insertar captura real de admin/maquinas.html mostrando lavadoras/secadoras, estado, crédito, tiempo restante y acciones.]

Fuente: Elaboración propia.

Figura 9. *Captura del panel de caja e informes*

[PENDIENTE: insertar captura real de admin/caja.html o admin/informes.html mostrando ingresos, ciclos y evolución.]

Fuente: Elaboración propia.

2.4. Concepción de la base de datos (MER, diagrama lógico de tablas, etc.)

La base de datos se denomina kwl_lavanderia y está diseñada sobre MariaDB con codificación utf8mb4. Su objetivo es almacenar la información persistente del sistema: lavanderías, usuarios, relación usuario-lavandería, máquinas, tarifas, ciclos, movimientos económicos, logs técnicos, auditoría y configuración. El

modelo se ha definido para conservar histórico y trazabilidad, especialmente en los cambios de tarifa y en los ciclos ya ejecutados.

El diseño distingue entre estado histórico y estado operativo. MariaDB contiene la verdad persistente: ciclos, movimientos, logs y auditoría. Redis se reserva para estado rápido y caché, pero no sustituye a la base de datos. Esta separación evita perder información contable o técnica si la caché se reinicia.

Figura 4. Modelo entidad-relación del sistema

[PENDIENTE: insertar el archivo “Diagrama - MER - Relleno.drawio” exportado a PNG/PDF. Debe aparecer en tamaño legible.]

Fuente: Elaboración propia.

Tabla 9. Diagrama lógico de tablas

Tabla	Atributos principales
lavanderia	id_lavanderia, nombre, codigo, direccion, ciudad, provincia, activo, fecha_alta, fecha_actualizacion
usuario	id_usuario, nombre, apellidos, login, password_hash, rol, activo, fecha_creacion, ultimo_acceso, fecha_actualizacion
usuario_lavanderia	id_usuario_lavanderia, id_usuario, id_lavanderia, fecha_alta
maquina	id_maquina, id_lavanderia, codigo_visible, tipo_maquina, estado_actual, activa, fecha_alta, observaciones, fecha_actualizacion
tarifa_maquina	id_tarifa, id_lavanderia, nombre, precio_arranque, tiempo_base_minutos, importe_incremento, minutos_por_incremento, fecha_inicio_vigencia, fecha_fin_vigencia, activa, fecha_creacion, fecha_actualizacion
ciclo	id_ciclo, id_maquina, id_tarifa_aplicada, fecha_hora_inicio, fecha_hora_fin, estado_ciclo, precio_arranque_aplicado, tiempo_base_aplicado_min, minutos_extra_total, importe_cliente_total, importe_bonificado_total, importe_total_aplicado, duracion_total_programada_min, observaciones, fecha_creacion, fecha_actualizacion
movimiento_maquina	id_movimiento, id_lavanderia, id_maquina, id_ciclo, id_usuario, fecha_hora, tipo_movimiento, origen_movimiento, importe, minutos_extra_generados, es_bonificacion, descripcion, fecha_creacion
log_maquina	id_log, id_lavanderia, id_maquina, id_ciclo, fecha_hora, tipo_evento, nivel, payload, procesado, fecha_creacion
auditoria	id_auditoria, id_usuario, id_lavanderia, id_maquina, id_ciclo, fecha_hora, accion, entidad_afectada, id_entidad_afectada, detalle, ip_origen, fecha_creacion
configuracion	id_configuracion, ambito, id_lavanderia, clave, valor, descripcion, fecha_actualizacion

Fuente: Elaboración propia.

Las relaciones principales son: una lavandería tiene muchas máquinas y muchas tarifas; un usuario puede estar asociado a una o varias lavanderías; una máquina puede tener muchos ciclos; un ciclo conserva la tarifa aplicada y puede tener múltiples movimientos; los logs técnicos se relacionan con lavandería, máquina y ciclo cuando procede; y la auditoría permite vincular acciones administrativas con usuario, lavandería, máquina o ciclo.

Tabla 10. Relaciones y reglas de integridad principales

Relación	Regla	Motivo
lavanderia 1 - N maquina	maquina.id_lavanderia referencia a	Cada máquina pertenece a un local

Relación	Regla	Motivo
	lavanderia.id_lavanderia	concreto.
lavanderia 1 - N tarifa_maquina	tarifa_maquina.id_lavanderia referencia a lavanderia.id_lavanderia	Las tarifas se definen por local y periodo.
usuario N - N lavanderia	usuario_lavanderia actúa como tabla puente	Permite usuarios con acceso a una o varias lavanderías.
maquina 1 - N ciclo	ciclo.id_maquina referencia a maquina.id_maquina	Cada ciclo pertenece a una máquina.
tarifa_maquina 1 - N ciclo	ciclo.id_tarifa_aplicada referencia a tarifa_maquina.id_tarifa	El ciclo conserva la tarifa histórica aplicada.
ciclo 1 - N movimiento_maquina	movimiento_maquina.id_ciclo referencia a ciclo.id_ciclo	Permite registrar arranques y ampliaciones.
maquina 1 - N log_maquina	log_maquina.id_maquina referencia a maquina.id_maquina	Mantiene histórico técnico por máquina.
usuario 1 - N auditoria	auditoria.id_usuario referencia a usuario.id_usuario	Permite saber quién ejecutó una acción.

Fuente: Elaboración propia.

Una decisión importante del modelo es que el ciclo guarda copia de los importes aplicados en el momento del uso: precio de arranque, tiempo base, minutos extra, importe de cliente, importe bonificado e importe total aplicado. Así, si se cambia una tarifa en el futuro, los ciclos anteriores no quedan alterados. Este criterio es fundamental para caja, informes y auditoría.

2.5. Diseño y arquitectura de la solución (módulos, patrones y/o estrategias utilizadas en la programación, diseño de clases si procede, ayudas concebidas, manipulación de errores, etc.)

La solución se ha diseñado como una arquitectura distribuida basada en eventos. El principio central es: el servidor decide y el dispositivo ejecuta. El frontend no accede al hardware ni a la base de datos directamente; toda acción pasa por la API. El backend valida permisos, aplica reglas de negocio, registra trazabilidad y publica comandos MQTT. Los dispositivos físicos o simulados reciben comandos, ejecutan acciones y publican estados o eventos.

Figura 1. Diagrama de red del sistema KWL

[PENDIENTE: insertar el archivo “Diagrama - Red - Relleno.drawio” exportado a PNG/PDF.]

Fuente: Elaboración propia.

Figura 2. Diagrama UML de componentes

[PENDIENTE: insertar el archivo “Diagrama - UML_Component - Relleno.drawio” exportado a PNG/PDF.]

Fuente: Elaboración propia.

Figura 3. Diagrama UML de despliegue

[PENDIENTE: insertar el archivo “Diagrama - UML_Deployment - Relleno.drawio” exportado a PNG/PDF.]

Fuente: Elaboración propia.

A nivel lógico, el sistema se divide en los siguientes componentes: frontend público y administrativo; backend API; bridge MQTT; scheduler; base de datos MariaDB; caché Redis; broker Mosquitto; simulador; y futura capa de hardware real basada en ESP32. Esta separación facilita el mantenimiento, porque cada parte tiene una responsabilidad concreta.

Tabla 11. Estructura de carpetas del proyecto

Carpeta	Responsabilidad
context/	Fuente conceptual del sistema: dominio, API, base de datos, MQTT, reglas y alcance del MVP.
docs/	Documentación de apoyo sobre decisiones, desarrollo e integración con elementos del local.
app/backend/	Backend Node.js/TypeScript con API, autenticación, MQTT, WebSocket, Redis y base de datos.
app/frontend/	Interfaz web pública y panel administrativo.
app/db/	Modelo físico SQL de MariaDB.
simulation/	Simulador de máquinas y dispositivos IoT.
deploy/demo/	Despliegue completo de evaluación mediante Docker Compose.
deploy/real/	Base de despliegue real separando VM_CORE, VM_DATA y LXC_MQTT.

Fuente: Elaboración propia.

En el backend se aplican varias estrategias: rutas Express por módulo, middlewares de autenticación y autorización, variables de entorno para configuración, conexión a base de datos centralizada, uso de Helmet para cabeceras HTTP, CORS controlado, hash de contraseñas, bridge MQTT, scheduler de tareas IoT y WebSocket para snapshots de panel administrativo.

Tabla 12. Resumen de módulos de la API REST

Módulo	Métodos	Función	Ejemplos de rutas internas
/api/auditoria	GET	Consulta de acciones registradas.	/
/api/auth	GET, POST	Autenticación, cierre de sesión y usuario actual.	/login, /logout, /me
/api/caja	GET	Consulta económica por periodos.	/dia, /semana, /rango, /mensual, /anual
/api/camera	GET, POST	Proxy, control y visualización de cámara.	/ptz/status, /ptz/center, /zoom, /display-mode, /relay/pulse
/api/configuracion	GET, PUT	Configuración general, web pública, entorno y tarifas.	/, /env, /web-public, /web-public/admin, /web-public/admin
/api/dashboard	GET	Resumen y estado operativo.	/resumen, /tiempo-real
/api/eventos	GET	Consulta básica de eventos.	/
/api/informes	GET	Informes de ciclos, evolución y tramos horarios.	/ciclos, /evolucion/semanal, /evolucion/mensual, /evolucion/anual, /tramos/diario
/api/iot	GET, POST, PUT	Estado, relés, horarios y escenas de domótica.	/state, /approx-state, /relay-action-log, /relay-action, /state
/api/lavanderias	GET, POST	Gestión de lavanderías.	/, /
/api/maquinas	GET, POST, PUT	Consulta y control de máquinas.	/, /:id, /:id/ventilador-auto, /:id/iniciar, /:id/detener

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Módulo	Métodos	Función	Ejemplos de rutas internas
/api/tienda	POST	Acciones generales de apertura/cierre.	/abrir, /cerrar
/api/usuarios	DELETE, GET, POST, PUT	Gestión de usuarios y lavanderías asociadas.	/, /, /id, /id/activar, /id/desactivar
/api/version	GET	Información de versión del backend.	/

Fuente: Elaboración propia.

La comunicación MQTT utiliza topics con prefijo kwl. El backend se suscribe a estados y eventos de máquinas y dispositivos IoT. Cuando un administrador realiza una acción, el backend publica un comando. Esta estrategia permite sustituir el simulador por un ESP32 real sin rediseñar la API web.

Tabla 13. *Topics MQTT principales*

Topic	Emisor	Receptor	Uso
kwl/maquinas/{codigo}/estado	ESP32 o simulador	Backend	Publicar estado de máquina sin identificador de lavandería.
kwl/maquinas/{codigo}/evento	ESP32 o simulador	Backend	Publicar eventos de máquina: inicio, fin, moneda, ampliación, error.
kwl/maquinas/{idLav}/{codigo}/estado	ESP32 o simulador	Backend	Publicar estado diferenciando lavandería.
kwl/maquinas/{idLav}/{codigo}/evento	ESP32 o simulador	Backend	Publicar eventos diferenciando lavandería.
kwl/maquinas/{idLav}/{codigo}/comando	Backend	ESP32 o simulador	Enviar comandos de crédito, ventilador, inicio o acciones físicas.
kwl/iot/{idLav}/estado	ESP32 o simulador	Backend	Sincronizar puerta, luces y ventilación.
kwl/iot/{idLav}/evento	ESP32 o simulador	Backend	Publicar eventos técnicos de dispositivos IoT.
kwl/iot/{idLav}/comando	Backend	ESP32 o simulador	Enviar órdenes globales de tienda.
kwl/config/{idLav}/tarifa	Backend	ESP32 o simulador	Publicar tarifa activa retenida para que el dispositivo conozca precio y tiempos.

Fuente: Elaboración propia.

El modelo de estados de máquina contempla estados persistidos como STOP, EN_MARCHA, PAUSADA, FUERA_SERVICIO y MANTENIMIENTO. Además, se manejan estados operativos auxiliares como crédito acumulado, segundos restantes, puerta cerrada, apertura pendiente o puerta abierta. Estos datos permiten que el panel muestre información útil sin mezclar estado histórico con estado runtime.

Figura 5. *Diagrama de estados de máquina*

[PENDIENTE: insertar el archivo “Diagrama - Estados - Relleno.drawio” exportado a PNG/PDF.]

Fuente: Elaboración propia.

Figura 6. *Diagrama de secuencia de concesión de crédito*

[PENDIENTE: insertar el archivo “Diagrama - Secuencia - Relleno.drawio”]

exportado a PNG/PDF.]

Fuente: Elaboración propia.

El flujo típico de crédito funciona así: el administrador pulsa la acción en la web; el frontend llama a la API; el backend valida autenticación, rol y lavandería; registra la acción en auditoría o movimiento; publica un comando MQTT; el dispositivo ejecuta la acción; el dispositivo publica evento o estado; el backend procesa la respuesta, actualiza base de datos y estado operativo; y el panel muestra el cambio.

La gestión de errores se ha diseñado en diferentes niveles. En HTTP se devuelven respuestas JSON con ok=false y códigos de error cuando hay credenciales incorrectas, permisos insuficientes, máquina inexistente o datos inválidos. En MQTT se controlan conexión, reconexión y parseo de mensajes. En base de datos se utilizan restricciones, claves foráneas e índices para evitar inconsistencias. En interfaz se recomienda mostrar mensajes claros y bloquear acciones críticas cuando la máquina esté offline o el usuario no tenga rol suficiente.

En materia de seguridad, el sistema sigue varias líneas: autenticación con token, roles ADMIN/OPERADOR/TÉCNICO, comprobación de acceso a lavandería, hash de contraseñas, Helmet, CORS restringido, variables de entorno, separación entre entorno demo y real, no publicación de contraseñas reales en la memoria, y acceso remoto preferentemente mediante VPN. Estas decisiones están alineadas con buenas prácticas de seguridad web y verificación de controles técnicos (OWASP Foundation, s. f.).

2.6. Principales dificultades encontradas en el desarrollo del proyecto y cómo se han afrontado.

La primera dificultad fue transformar un problema real de hardware y negocio en un alcance académico viable. El sistema existente es cerrado, utiliza placas personalizadas y no proporciona acceso total al software ni a la base de datos. Para afrontarlo, el proyecto se planteó como sustitución progresiva: primero se valida la arquitectura con simulador y después se prepara la integración física con ESP32.

La segunda dificultad fue interpretar el funcionamiento físico sin disponer de toda la documentación del proveedor. La consultoría indica presencia de señales a 24 V, relés, pulsos, monedero RM5, placas MPS1 y comunicación por RJ45. Dado que no se debe asumir que todas las señales son accesibles o seguras, la memoria diferencia claramente entre lo implementado en software y lo que requiere validación eléctrica antes de producción.

La tercera dificultad fue diseñar la comunicación entre web y dispositivos. Un enfoque directo HTTP desde el frontend al dispositivo habría sido inseguro y difícil de mantener. La solución adoptada fue usar el backend como centro de decisión y MQTT como canal de comunicación con dispositivos. Así, el frontend no conoce ni controla hardware directamente.

La cuarta dificultad fue probar sin hardware real constante. Para resolverlo se desarrolló un simulador que representa máquinas y dispositivos, publica estados y eventos, recibe comandos y permite validar el flujo completo. Esto ha sido clave para demostrar el proyecto en entorno académico sin depender de instalar placas en todas las máquinas reales.

La quinta dificultad fue mantener coherencia entre contabilidad, ciclos y tarifas. Si una tarifa cambia, los ciclos antiguos no deben variar. Por eso el modelo de base de datos guarda los valores aplicados dentro de cada ciclo y registra movimientos económicos con origen MONEDERO o WEB_MANUAL. Esta decisión permite informes más fiables y evita alterar el histórico.

La sexta dificultad fue organizar un proyecto grande sin que quedara disperso. La estructura separa contexto documental, backend, frontend, base de datos, simulación y despliegue. Además, los diagramas draw.io ayudan a explicar red, componentes, despliegue, base de datos, estados y secuencia. En la memoria se usan como puntos de apoyo para que el tribunal pueda entender la solución global.

La séptima dificultad fue el despliegue. No basta con ejecutar código local: el sistema requiere MariaDB, Redis, MQTT, backend, frontend, simulador y proxy Nginx. Docker Compose permite levantar todo el entorno demo de forma reproducible. Para el entorno real, se propone separación en Proxmox entre VM_CORE, VM_DATA y LXC_MQTT.

La octava dificultad fue la seguridad de acciones críticas. Apagar una máquina, conceder crédito o abrir una puerta no puede estar disponible para cualquier usuario. Por ello se utilizan middlewares de autenticación, control de rol y control de lavandería, además de dejar recomendada la operación remota por VPN y no exponer servicios internos directamente a Internet.

2.7. Despliegue (dónde está alojada, cómo acceder, qué se necesita, etc.)

El proyecto contempla dos formas de despliegue: entorno demo/evaluación y entorno real. El entorno demo está pensado para la defensa del TFC y permite ejecutar todo en una sola máquina mediante Docker Compose. El entorno real está pensado para una lavandería física y separa servicios por seguridad, mantenimiento y escalabilidad.

El entorno demo utiliza los siguientes servicios: MariaDB, Redis, Eclipse Mosquitto, simulador MQTT, interfaz GUI del simulador, backend Node.js, Nginx para frontend/backend y Adminer para base de datos. Esta configuración permite iniciar el sistema completo y probar máquinas simuladas, eventos, caja, informes y panel web.

Tabla 14. *Servicios del despliegue demo*

Servicio	Imagen/tecnología	Puerto publicado	Función
mariadb	mariadb:11.4	3307 -> 3306	Base de datos kwl_lavanderia.
redis	redis:7-alpine	127.0.0.1:6379	Caché operativa y soporte

Servicio	Imagen/tecnología	Puerto publicado	Función
			de estado.
mqtt	eclipse-mosquitto:2	127.0.0.1:1883	Broker MQTT.
mqtt-sim	simulation/Dockerfile	interno	Simulador automático de máquinas.
mqtt-sim-gui	simulation/Dockerfile	8083	Interfaz gráfica del simulador.
core-node	backend/Dockerfile	interno	API backend Node.js/TypeScript.
core-nginx	nginx:1.27-alpine	8080 y 8081	Proxy backend y servidor frontend.
adminer	adminer:4	8082	Inspección de base de datos en demo.

Fuente: Elaboración propia.

Para ejecutar el entorno demo se recomienda usar el archivo `deploy/demo/docker-compose.yml`. En Windows, el proyecto incluye `Launcher.bat` y scripts PowerShell para simplificar el arranque. En Linux/Fedora se incluye un script de despliegue específico. El procedimiento manual básico es el siguiente:

```
cd deploy/demo
copy .env.example .env      # En Windows
# cp .env.example .env      # En Linux/macOS
docker compose up -d --build
```

Después del arranque, los accesos previstos para evaluación son:

- Frontend/admin mediante Nginx: `http://127.0.0.1:8081` [PENDIENTE: confirmar ruta final exacta en la defensa].
- API/health: `http://127.0.0.1:8080/health`.
- Adminer: `http://127.0.0.1:8082`.
- GUI del simulador: `http://127.0.0.1:8083`.
- Broker MQTT local: `127.0.0.1:1883`, no expuesto públicamente.

Figura 10. *Captura del simulador IoT/MQTT*

[PENDIENTE: insertar captura de la GUI del simulador en `http://127.0.0.1:8083` mostrando máquinas y eventos.]

Fuente: Elaboración propia.

Figura 11. *Captura del despliegue Docker Compose*

[PENDIENTE: insertar captura de docker compose ps con los servicios en estado Up/healthy.]

Fuente: Elaboración propia.

Figura 12. *Captura de comprobación de health del backend*

[PENDIENTE: insertar captura de `/health` mostrando db, redis, mqtt e iot.]

Fuente: Elaboración propia.

El entorno real se plantea sobre Proxmox VE. La separación propuesta es VM_CORE para backend, frontend, Redis y tiempo real; VM_DATA para MariaDB; y LXC_MQTT para Mosquitto. Esta separación evita que todos los componentes críticos dependan de un único contenedor y facilita hacer copias, reinicios y mantenimiento de forma independiente. El acceso administrativo remoto debe realizarse mediante VPN WireGuard o solución equivalente, evitando exponer directamente la base de datos, el broker MQTT o paneles internos.

Credenciales de prueba: [PENDIENTE: incluir usuario administrador ficticio para evaluación, sin contraseñas reales de producción]. Se recomienda usar una contraseña temporal exclusivamente para la defensa y cambiarla o eliminarla posteriormente.

2.8. Validación y pruebas

La validación del proyecto debe demostrar que los requisitos principales funcionan de forma controlada. Se propone conservar evidencias mediante capturas de pantalla, logs de consola y capturas del panel. Las pruebas se han organizado por áreas: autenticación, máquinas, MQTT, base de datos, caja, informes, domótica, simulador y despliegue.

Tabla 15. Plan de pruebas funcionales

Prueba	Pasos	Resultado esperado	Evidencia
Health del backend	Abrir /health con contenedores levantados.	Respuesta JSON con servicio activo y estado de DB/Redis/MQTT.	[PENDIENTE: captura].
Login correcto	Entrar con usuario administrador de prueba.	Acceso al panel administrativo.	[PENDIENTE: captura].
Login incorrecto	Introducir credenciales inválidas.	Mensaje de error y sin acceso al panel.	[PENDIENTE: captura].
Listado de máquinas	Abrir panel Máquinas.	Aparecen lavadoras/secadoras de la lavandería.	[PENDIENTE: captura].
Concesión de crédito	Aplicar crédito desde la web a una máquina simulada.	Se registra movimiento/evento y el simulador recibe comando.	[PENDIENTE: captura/log].
Inicio de ciclo	Iniciar máquina desde simulador o web.	Estado EN_MARCHA y ciclo iniciado en BD.	[PENDIENTE: captura BD/panel].
Finalización de ciclo	Esperar fin simulado o forzar evento.	Estado PAUSADA/STOP y ciclo finalizado.	[PENDIENTE: captura].
Ampliación de secadora	Aplicar incremento durante ciclo de secadora.	Se añaden minutos e importe según tarifa.	[PENDIENTE: captura].
Caja diaria	Consultar panel caja.	Importes calculados desde movimientos/ciclos.	[PENDIENTE: captura].
Logs	Abrir panel logs tras acciones.	Se observan eventos técnicos y auditoría.	[PENDIENTE: captura].
Domótica	Activar/desactivar puerta, luces o ventilación simulada.	Estado IoT sincronizado por MQTT.	[PENDIENTE: captura].
Permisos	Intentar acción admin con usuario sin rol.	Respuesta de no autorizado/prohibido.	[PENDIENTE: captura].

Fuente: Elaboración propia.

El resultado de las pruebas debe completarse antes de la entrega definitiva. Esta tabla está preparada para que se añadan las evidencias finales. En la defensa conviene mostrar al menos el despliegue, el login, el panel de máquinas, el simulador, una acción MQTT y el registro resultante en logs o base de datos.

3. Conclusiones y posibles ampliaciones

El proyecto LAVANDERÍA KWL ha permitido construir la base de un sistema integral para una lavandería autoservicio real. El trabajo no se limita a resolver un ejercicio web aislado, sino que aborda un problema de negocio: dependencia de un proveedor cerrado, falta de acceso a datos, dificultad de mantenimiento y necesidad de controlar máquinas y dispositivos del local desde una solución propia.

El resultado principal es un MVP funcional y documentado que demuestra la arquitectura completa: frontend, backend, base de datos, MQTT, Redis, simulador y despliegue. Esta arquitectura permite operar máquinas simuladas, registrar ciclos, gestionar créditos, consultar caja, visualizar informes, controlar elementos IoT y mantener trazabilidad de acciones. Además, queda preparada para sustituir progresivamente partes del sistema físico mediante ESP32 u otros módulos compatibles.

A nivel formativo, el proyecto integra muchas competencias del ciclo: desarrollo frontend, backend, base de datos, API REST, seguridad, despliegue, redes, documentación, análisis de requisitos y diseño de sistemas. También obliga a tomar decisiones de ingeniería, como separar estado operativo e histórico, no exponer servicios internos, registrar auditoría, usar simulación para pruebas y diseñar una arquitectura desacoplada.

Tabla 16. Cumplimiento de objetivos

Objetivo	Estado	Comentario
Sistema web administrativo	Cumplido	Existe frontend público y panel administrativo por módulos.
Backend central	Cumplido	API Node.js/TypeScript con rutas, autenticación, configuración, caja, informes, usuarios y máquinas.
Base de datos relacional	Cumplido	Modelo MariaDB con tablas de lavandería, usuario, máquina, tarifa, ciclo, movimiento, logs, auditoría y configuración.
Comunicación MQTT	Cumplido	Bridge MQTT con topics para máquinas, IoT, comandos, estados, eventos y tarifas.
Simulador de dispositivos	Cumplido	Simulador de máquinas y dispositivos con interfaz y MQTT.
Despliegue demo	Cumplido	Docker Compose levanta servicios necesarios para evaluación.
Infraestructura real	Parcial	Diseñada para Proxmox con VM/LXC; requiere validación final en instalación real.
Integración física ESP32	Parcial/futura	Arquitectura preparada; firmware físico definitivo requiere pruebas eléctricas y de campo.
Pagos online	Futuro	No incluido en MVP.
IA/mantenimiento predictivo	Futuro	Requiere histórico suficiente de datos reales.

Fuente: Elaboración propia.

Las principales limitaciones actuales son la dependencia de pruebas físicas para sustituir placas reales, la falta de acceso interno al software de las lavadoras existentes, la necesidad de verificar señales eléctricas antes de conectar microcontroladores y la ausencia de pagos reales o app de cliente final. Estas limitaciones no invalidan el MVP; simplemente delimitan qué parte pertenece a la entrega académica y qué parte corresponde a una futura implantación productiva.

Las ampliaciones futuras más relevantes son: integración física completa con ESP32, pagos online mediante QR o pasarela, área de cliente, notificaciones por Telegram/WhatsApp, predicción de mantenimiento, sensores ambientales, gestión multi-local, copias de seguridad automáticas, monitorización avanzada, dashboard económico más detallado y endurecimiento de seguridad para producción.

Como conclusión personal, el proyecto demuestra que una solución propia y abierta puede ofrecer al propietario más control, más trazabilidad y mayor capacidad de evolución que un sistema cerrado. El diseño por capas permite que cada componente pueda mantenerse o sustituirse sin rehacer todo el sistema, que es precisamente una de las necesidades principales detectadas al inicio del proyecto.

4. Bibliografía y webgrafía

Las siguientes referencias se presentan en formato APA 7. Las fuentes internas corresponden a documentos de análisis, entrevista, requisitos y consultoría utilizados para definir el alcance del proyecto. Las fuentes web corresponden principalmente a documentación oficial de tecnologías empleadas.

Departamento de Informática. (2026). Criterios de calificación del módulo de Proyecto [Rúbrica interna].

Docker. (s. f.). Docker Compose. Recuperado el 10 de mayo de 2026, de <https://docs.docker.com/compose/>

Eclipse Foundation. (s. f.). Eclipse Mosquitto. Recuperado el 10 de mayo de 2026, de <https://mosquitto.org/>

Espressif Systems. (s. f.). ESP32: A feature-rich MCU with integrated Wi-Fi and Bluetooth connectivity. Recuperado el 10 de mayo de 2026, de <https://www.espressif.com/en/products/socs/esp32>

MariaDB. (s. f.). MariaDB Server documentation. Recuperado el 10 de mayo de 2026, de <https://mariadb.com/docs/server>

Microsoft. (s. f.). TypeScript: JavaScript with syntax for types. Recuperado el 10 de mayo de 2026, de <https://www.typescriptlang.org/>

Node.js. (s. f.). Node.js. Recuperado el 10 de mayo de 2026, de <https://nodejs.org/en>

OASIS. (2019). MQTT version 5.0. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

OpenJS Foundation. (s. f.). Express: Node.js web application framework.

Recuperado el 10 de mayo de 2026, de <https://expressjs.com/>

OWASP Foundation. (s. f.). OWASP Application Security Verification Standard (ASVS). Recuperado el 10 de mayo de 2026, de <https://owasp.org/www-project-application-security-verification-standard/>

Proxmox Server Solutions GmbH. (s. f.). Proxmox Virtual Environment documentation. Recuperado el 10 de mayo de 2026, de <https://pve.proxmox.com/pve-docs/>

Redis. (s. f.). Redis documentation. Recuperado el 10 de mayo de 2026, de <https://redis.io/docs/latest/>

Rodríguez López, D. (2026a). Entrevista QWL [Documento interno no publicado].

Rodríguez López, D. (2026b). Requerimientos QWL [Documento interno no publicado].

Rodríguez López, D. (2026c). Consultoría QWL [Documento interno no publicado].

WireGuard. (s. f.). WireGuard: Fast, modern, secure VPN tunnel. Recuperado el 10 de mayo de 2026, de <https://www.wireguard.com/>

ANEXOS

Anexo I. Lista de elementos pendientes antes de la entrega

- Actualizar el índice automático y el índice de figuras con numeración real de páginas.
- Insertar las seis figuras draw.io exportadas a imagen/PDF: red, componentes, despliegue, MER, estados y secuencia.
- Añadir capturas reales de la interfaz pública, panel de máquinas, caja/informes, simulador, Docker Compose y /health.
- Completar el nombre del centro educativo y la fecha final de entrega.
- Confirmar credenciales de evaluación con usuario ficticio y contraseña temporal, sin exponer credenciales reales.
- Sustituir costes materiales pendientes por importes reales o aproximados de componentes seleccionados.
- Confirmar URL/puertos definitivos que se usarán durante la defensa.
- Exportar la versión definitiva a PDF al menos cinco días lectivos antes de la defensa, según la rúbrica.

Anexo II. Guion recomendado de defensa oral

Duración objetivo: 10 minutos. Se recomienda estructurar la defensa en cinco bloques: problema, solución, arquitectura, demostración y conclusiones.

Tabla 17. *Guion de defensa oral*

Minuto	Contenido
0:00-1:00	Presentar el problema real: sistema propietario,

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Minuto	Contenido
	dependencia externa y necesidad de control.
1:00-2:00	Explicar objetivos y alcance del MVP.
2:00-4:00	Mostrar arquitectura: frontend, backend, MariaDB, Redis, MQTT, simulador y despliegue.
4:00-7:00	Demostración: login, máquinas, simulador, acción MQTT, logs y caja/informes.
7:00-8:30	Base de datos y trazabilidad: ciclos, movimientos, logs, auditoría y tarifas históricas.
8:30-9:30	Dificultades: sistema cerrado, hardware, simulación, despliegue y seguridad.
9:30-10:00	Conclusión y ampliaciones futuras.

Fuente: Elaboración propia.

Anexo III. Endpoints principales detectados en el código**Tabla 18. Endpoints principales de la API**

Ruta	Método	Uso
/api/auditoria	GET	Consulta de acciones registradas.
/api/auth/login	POST	Autenticación, cierre de sesión y usuario actual.
/api/auth/logout	POST	Autenticación, cierre de sesión y usuario actual.
/api/auth/me	GET	Autenticación, cierre de sesión y usuario actual.
/api/caja/dia	GET	Consulta económica por periodos.
/api/caja/semana	GET	Consulta económica por periodos.
/api/caja/rango	GET	Consulta económica por periodos.
/api/caja/mensual	GET	Consulta económica por periodos.
/api/caja/anual	GET	Consulta económica por periodos.
/api/camera/ptz/status	GET	Proxy, control y visualización de cámara.
/api/camera/ptz/center	POST	Proxy, control y visualización de cámara.
/api/camera/zoom	POST	Proxy, control y visualización de cámara.
/api/camera/display-mode	POST	Proxy, control y visualización de cámara.
/api/camera/relay/pulse	POST	Proxy, control y visualización de cámara.
/api/camera/stream.jpg	GET	Proxy, control y visualización de cámara.
/api/camera/faststream.mjpg	GET	Proxy, control y visualización de cámara.
/api/camera/ui/target	GET	Proxy, control y visualización de cámara.
/api/configuracion	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/env	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/web-public	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/web-public/admin	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/web-public/admin	PUT	Configuración general, web pública, entorno y tarifas.
/api/configuracion/env	PUT	Configuración general, web pública, entorno y tarifas.

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Ruta	Método	Uso
/api/configuracion/tarifa-actual	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/tarifa-actual	PUT	Configuración general, web pública, entorno y tarifas.
/api/configuracion/:clave	GET	Configuración general, web pública, entorno y tarifas.
/api/configuracion/:clave	PUT	Configuración general, web pública, entorno y tarifas.
/api/dashboard/resumen	GET	Resumen y estado operativo.
/api/dashboard/tiempo-real	GET	Resumen y estado operativo.
/api/eventos	GET	Consulta básica de eventos.
/api/informes/ciclos	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/evolucion/semanal	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/evolucion/mensual	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/evolucion/anual	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/tramos/diario	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/tramos/mensual	GET	Informes de ciclos, evolución y tramos horarios.
/api/informes/tramos/anual	GET	Informes de ciclos, evolución y tramos horarios.
/api/iot/state	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/approx-state	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/relay-action-log	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/relay-action	POST	Estado, relés, horarios y escenas de domótica.
/api/iot/state	PUT	Estado, relés, horarios y escenas de domótica.
/api/iot/schedule	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/schedule	PUT	Estado, relés, horarios y escenas de domótica.
/api/iot/store-schedule	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/store-schedule	PUT	Estado, relés, horarios y escenas de domótica.
/api/iot/store-actions	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/store-actions	PUT	Estado, relés, horarios y escenas de domótica.
/api/iot/store-open-machines	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/store-open-machines	PUT	Estado, relés, horarios y escenas de domótica.
/api/iot/store-close-machines	GET	Estado, relés, horarios y escenas de domótica.
/api/iot/store-close-machines	PUT	Estado, relés, horarios y escenas de domótica.

Ciclo Formativo de Grado Superior: DAW**LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA**

Ruta	Método	Uso
/api/iot/store/open	POST	Estado, relés, horarios y escenas de domótica.
/api/iot/store/close	POST	Estado, relés, horarios y escenas de domótica.
/api/lavanderias	GET	Gestión de lavanderías.
/api/lavanderias	POST	Gestión de lavanderías.
/api/maquinas	GET	Consulta y control de máquinas.
/api/maquinas/:id	GET	Consulta y control de máquinas.
/api/maquinas/:id/ventilador-auto	PUT	Consulta y control de máquinas.
/api/maquinas/:id/iniciar	POST	Consulta y control de máquinas.
/api/maquinas/:id/detener	POST	Consulta y control de máquinas.
/api/maquinas/:id/reiniciar	POST	Consulta y control de máquinas.
/api/maquinas/:id/credito	POST	Consulta y control de máquinas.
/api/maquinas/:id/ampliar	POST	Consulta y control de máquinas.
/api/tienda/abrir	POST	Acciones generales de apertura/cierre.
/api/tienda/cerrar	POST	Acciones generales de apertura/cierre.
/api/usuarios	GET	Gestión de usuarios y lavanderías asociadas.
/api/usuarios	POST	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id	PUT	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id/activar	POST	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id/desactivar	POST	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id/lavanderias	GET	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id/lavanderias	PUT	Gestión de usuarios y lavanderías asociadas.
/api/usuarios/:id	DELETE	Gestión de usuarios y lavanderías asociadas.
/api/version	GET	Información de versión del backend.

Fuente: Elaboración propia.

Anexo IV. Matriz de relación entre rúbrica y memoria**Tabla 19. Cumplimiento de apartados exigidos por la rúbrica**

Requisito de la rúbrica	Dónde se cubre en la memoria
Resumen	Sección RESUMEN.
Introducción y descripción del objeto de estudio	Sección 1.1.
Objetivos	Sección 1.2.
Estimación de costes y tiempo	Sección 2.1.
Tecnologías y herramientas utilizadas	Sección 2.2.
Diseño de interfaz	Sección 2.3.
Base de datos, MER y modelo lógico	Sección 2.4 y Figura 4.
Arquitectura de la solución	Sección 2.5 y Figuras 1, 2, 3, 5 y 6.
Dificultades y soluciones	Sección 2.6.

Ciclo Formativo de Grado Superior: DAW

LAVANDERÍA KWL: SISTEMA INTEGRAL DE GESTIÓN, AUTOMATIZACIÓN Y CONTROL DE UNA LAVANDERÍA

Requisito de la rúbrica	Dónde se cubre en la memoria
Despliegue	Sección 2.7.
Conclusiones y ampliaciones	Sección 3.
Bibliografía y webgrafía	Sección 4 en formato APA7.

Fuente: Elaboración propia.